

Optimized Implementation of Warp-Cooperative GPU HCTR2-ARIA Wide-Block Encryption

Siwoo Eum, Minho Song, Seungwon Lee, Hagyeong Kim, Hwajeong Seo

1. What is HCTR2?

2. Background: GPU and ARIA

3. Our Implementation Method

4. Performance and Conclusion

1. What is HCTR2 ?



SP 800-197

Wider Variant of AES – resolves the birthday-bound issue with a fixed large block size



SP 800-197A

Variable-length input adds flexibility (padding issue).
tweak values resolve key reuse



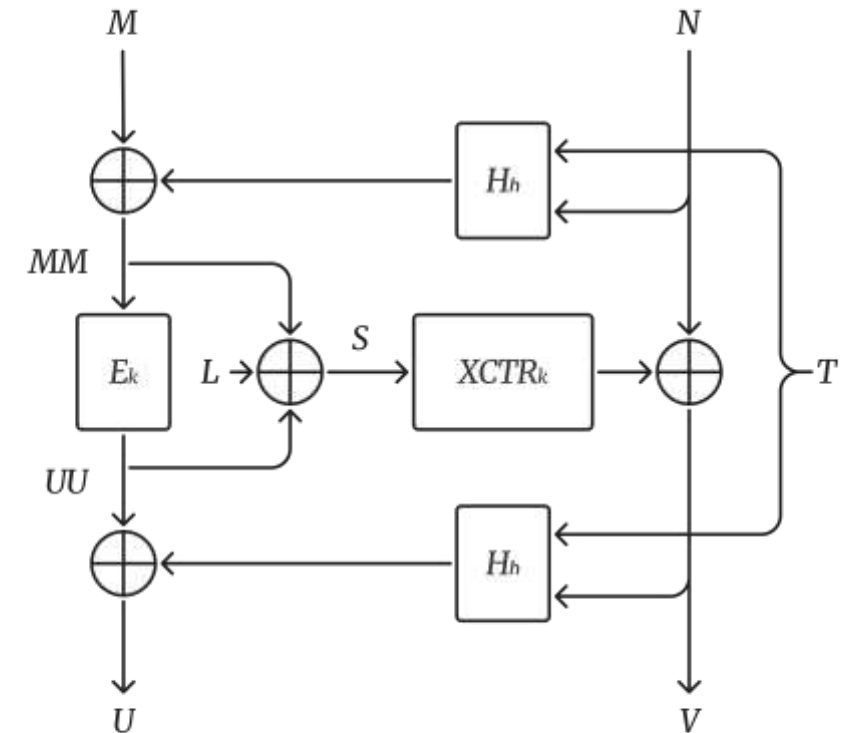
NIST plans three general-purpose **accordions block cipher mode**

- **Acc128** – built on the 128-bit block cipher (AES). single-key data expected to be capped near 2^{48} bits due to the birthday bound
- **Acc256** – uses a 256-bit block cipher (SP 800-197, Wider Variant of AES), so one key covers far more data
- **BBBAcc** – keeps the 128-bit block size but removes the birthday-bound limit, targeting Acc256-level data volumes with a single key

1. What is HCTR2 ?

Variable-Input-Length Strong Pseudorandom Permutation (VIL-SPRP)

- HCTR2 splits the data into **a fixed-size block M** and **a variable-length part N**, so that both halves influence the encryption process
- **N** is encrypted in XCTR mode, which produces a keystream of exactly matching length, so the ciphertext always keeps the original data length
- M depends on the encrypted N, and the final ciphertext U is mixed back with the encrypted V, ensuring integrity across the entire message



$$\text{XCTR}_k(S) = E_k(S \oplus \text{bin}(1)) \| E_k(S \oplus \text{bin}(2)) \| E_k(S \oplus \text{bin}(3)) \| \dots$$

1. What is HCTR2 ?

HCTR2

A tweakable wide-block cipher for length-preserving encryption of storage such as disk sectors



Cryptographic Accordion

NIST is standardizing a general-purpose Accordion mode based on the HCTR2 structure



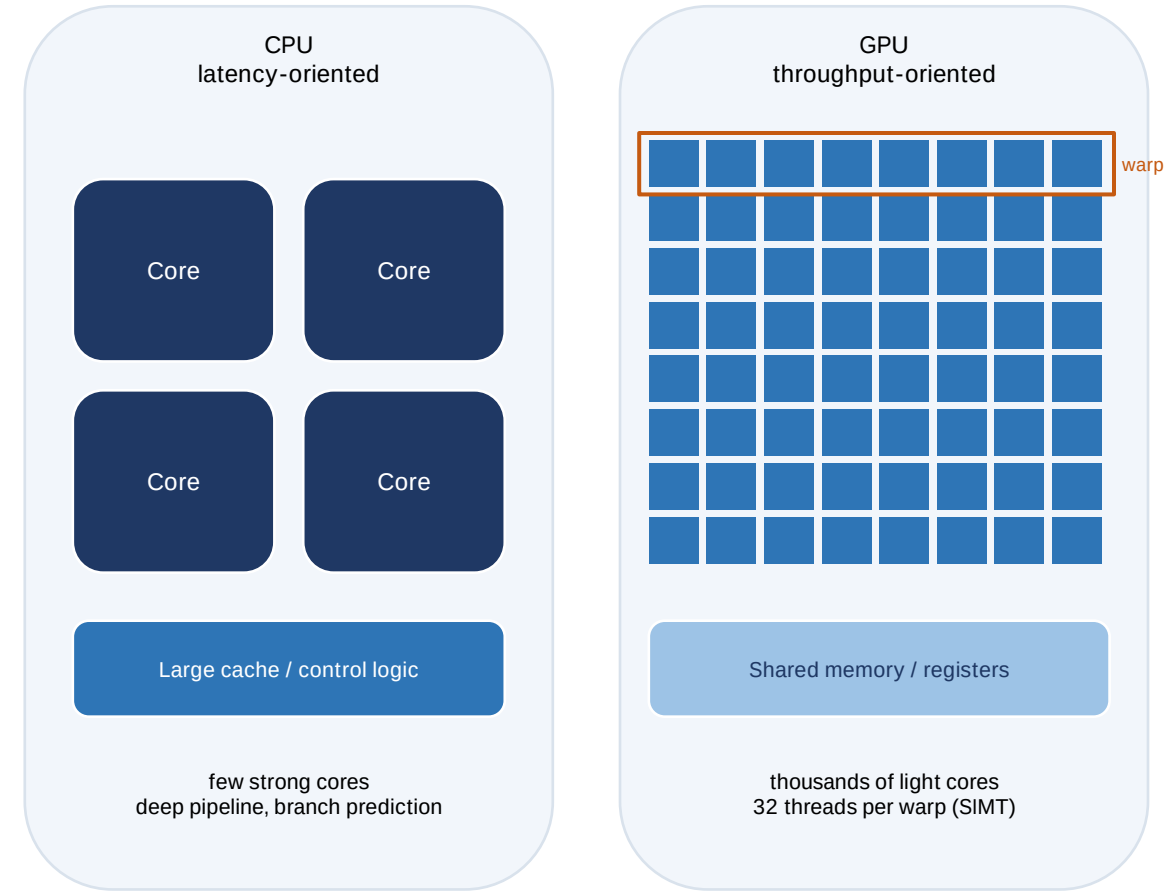
AEAD as a derived function

Applying input encoding to the HCTR2-based Accordion yields AEAD, addressing GCM's nonce-reuse weakness, nonce exposure and missing key commitment

HCTR2 suits length-preserving encryption of storage such as disk sectors. NIST is standardizing Accordion on top of the HCTR2 structure, and AEAD is then composed as a derived function of that Accordion through input encoding — HCTR2 itself does not become an AEAD.

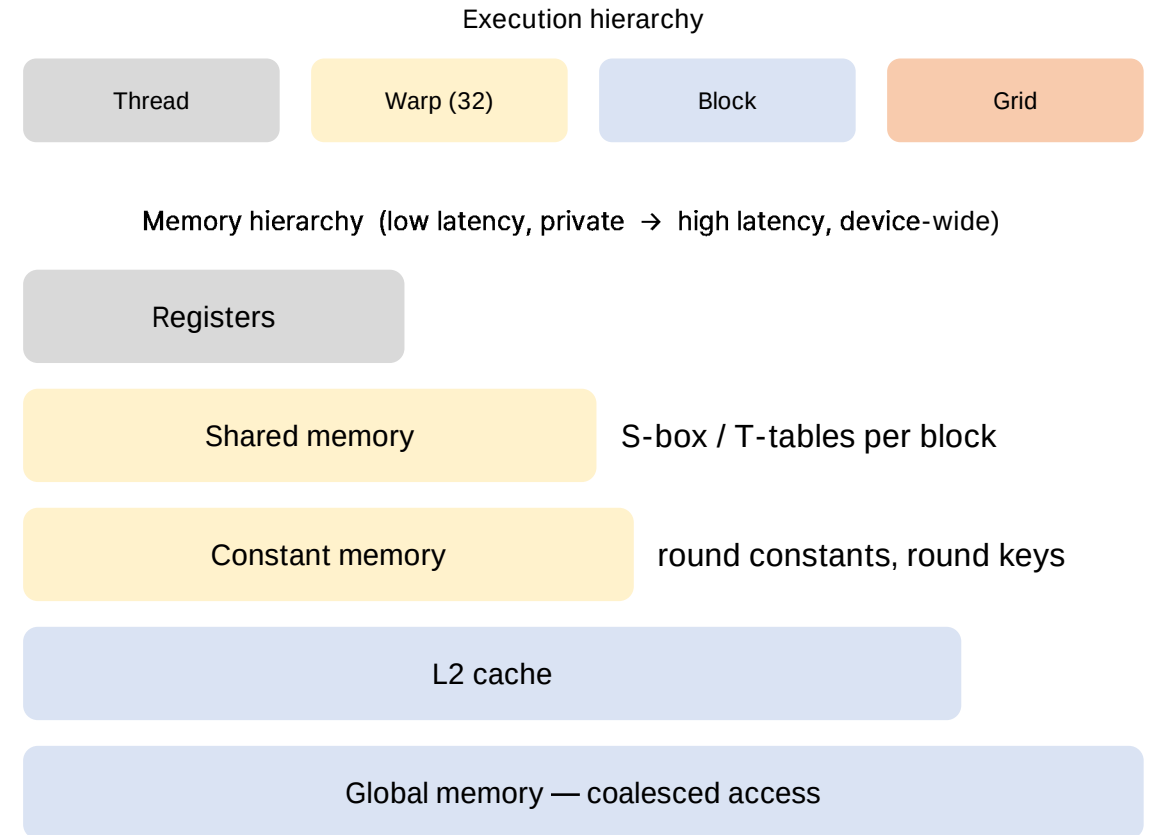
2. Background : GPU — CPU vs GPU for Cryptography

- CPU
 - latency-oriented
 - best for sequential, control-heavy code and single-message encryption
- GPU
 - throughput-oriented
 - thousands of lightweight cores in warps of 32 threads (SIMT)
 - best when one operation runs over massive independent data
- Why it fits block ciphers
 - XCTR is embarrassingly parallel across counter blocks, while HCTR2 needs threads to cooperate on one block



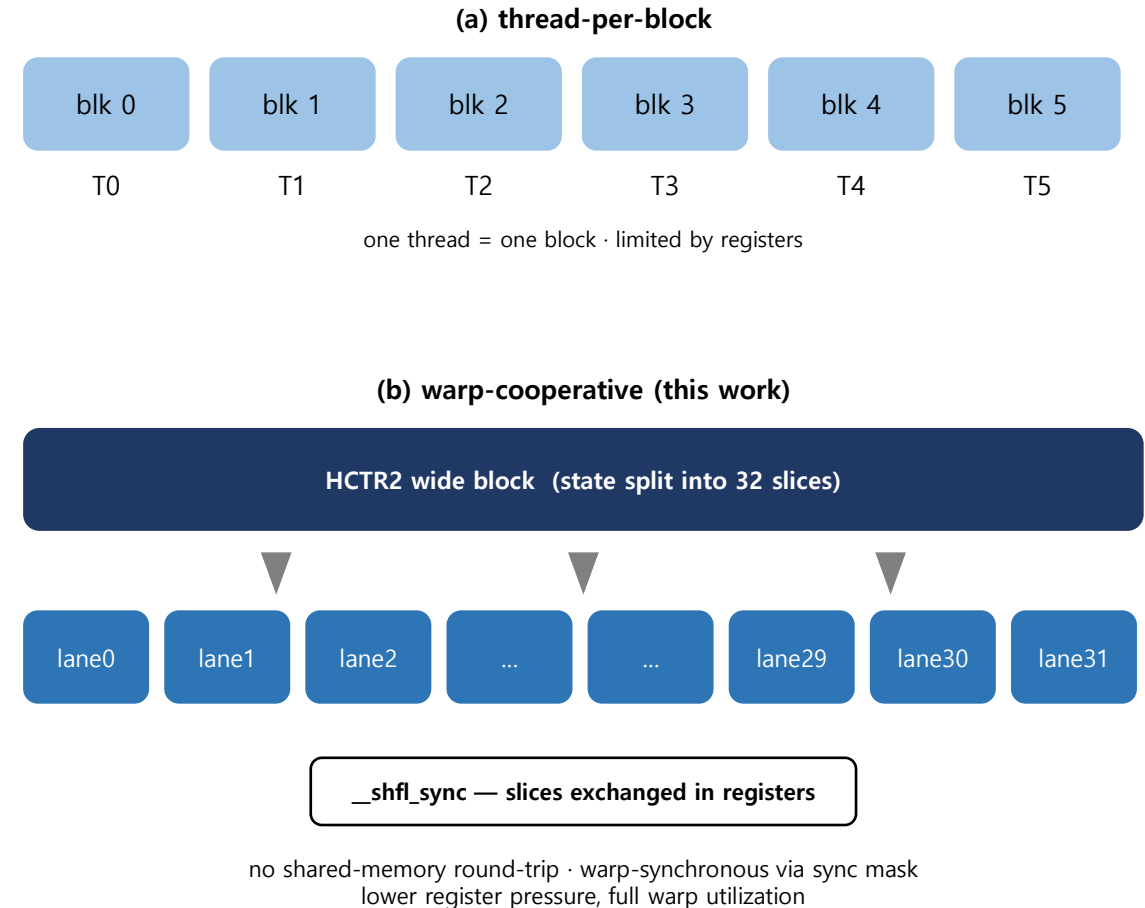
2. Background : GPU — Execution and Memory Model

- Execution hierarchy
 - thread → warp (32 threads) → block → grid
 - a warp issues one instruction for all its lanes (SIMT)
- On-chip memory
 - registers are private per thread
 - shared memory is per block
- Off-chip memory
 - constant memory
 - global memory



2. Background : GPU — Warp-Cooperative Processing

- Thread-per-block
 - each thread encrypts one independent block: simple, but the wide block of HCTR2 exceeds a single thread's register budget
- Warp-cooperative
 - 32 lanes share one wide block, each lane holding a slice of the state
- Exchange
 - `__shfl_sync` move slices directly in registers, so no shared-memory round-trip and no block-wide `__syncthreads`. the sync mask keeps all lanes in lockstep
- Benefits
 - lower register pressure per thread, better latency hiding, and full warp utilization on long messages



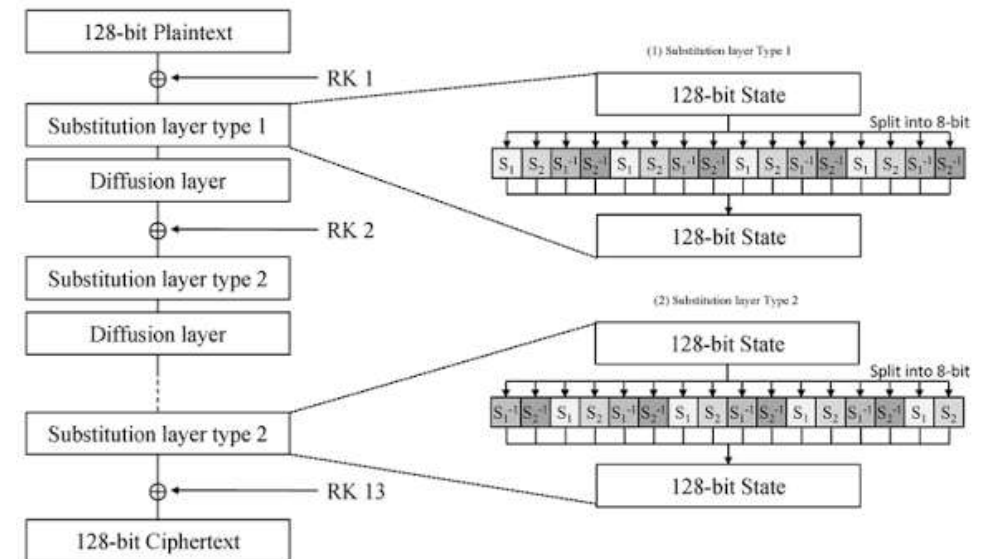
2. Background : ARIA — Algorithm Overview

Korea's national standard block cipher

- Adopted as a Korean national standard (KS X 1213) in 2004
- Involutional SPN structure — encryption and decryption share the same process
- Uses two S-boxes and a 16×16 binary diffusion matrix for strong security

Key Specifications

- Block size: 128 bit
- Key length: 128 / 192 / 256 bit
- Rounds: 12 / 14 / 16

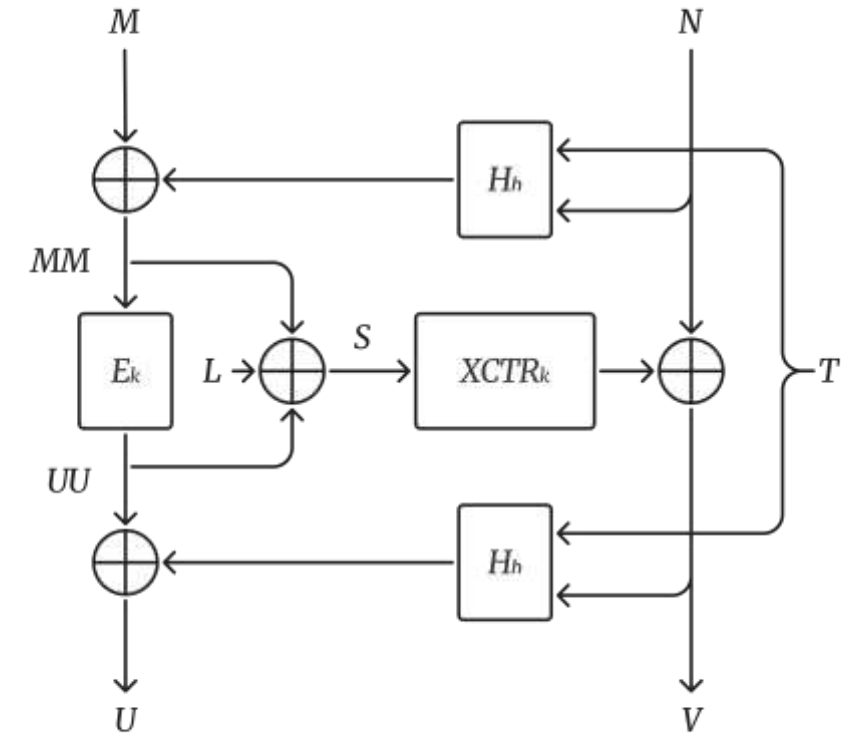


3. Our Implementation Method

16KiB

Operation	Calls	Characteristics
Central ARIA ECB	1	Encrypts the Hash 1 result
XCTR ARIA	1,023	Blocks are independent → parallelizable
POLYVAL GF multiply	~2,048	~1,024 each in Hash 1 and Hash 2

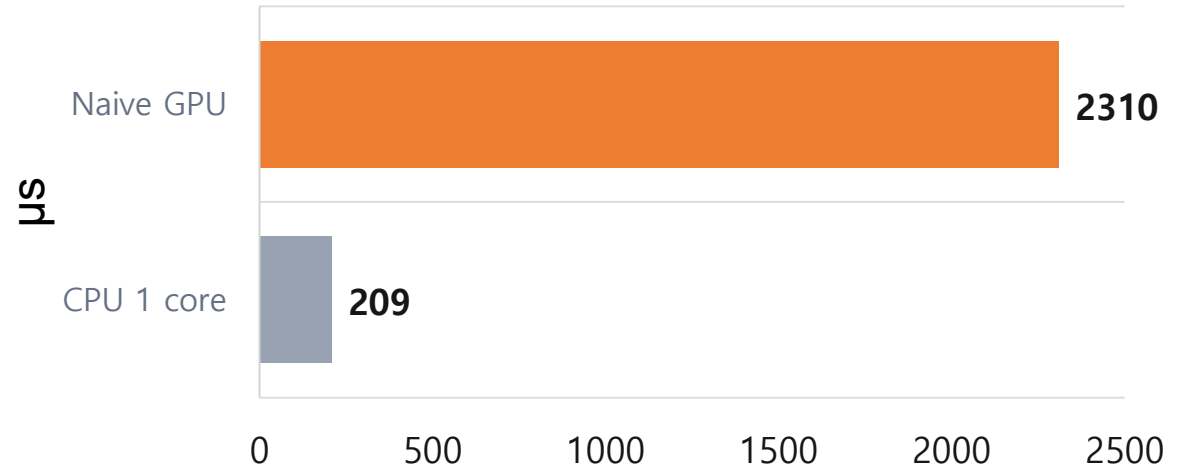
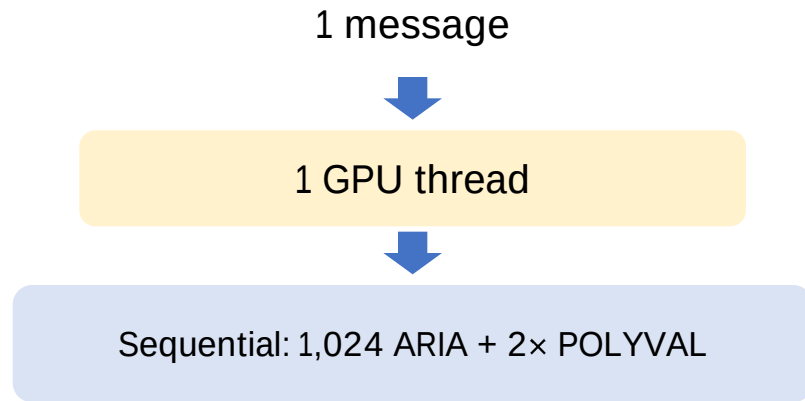
- XCTR's 1,023 ARIA operations are independent — ideal for GPU parallelization
- POLYVAL is a sequential hash chain: each block's result feeds the next
- Multi-message batching alone cannot reduce single-message latency
- POLYVAL and XCTR within one message must be split across lanes



The workload is sufficient. GPU performance depends on how POLYVAL's sequential chain is parallelized

3. Our Implementation Method

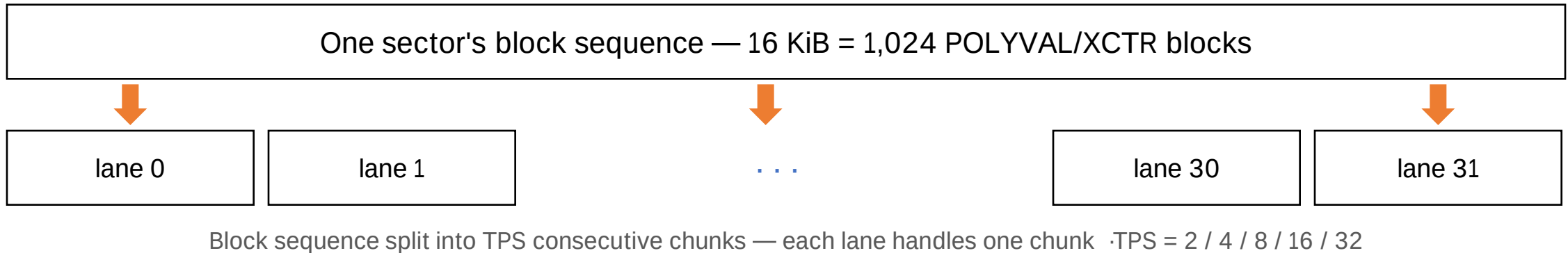
❖ Why the Naive GPU Implementation Is Slow for a Single Message



- With many concurrent messages throughput is high, but a single message forces one thread to run sequentially
- Of a warp's 32 lanes, effectively only one does the work while the rest stay idle
- The large HCTR2 state raises per-thread register usage, reducing concurrent warps and risking local-memory spill

3. Our Implementation Method

❖ Warp-Cooperative Mapping: One Message Shared Across Warp Lanes



- Baseline: one thread processes a sector's entire block sequence
→ Proposed: one warp handles a single sector
- The sector's internal block sequence (POLYVAL/XCTR) is split into TPS consecutive chunks, and each lane runs the ARIA POLYVAL operations for its chunk
- Each lane keeps only its own chunk's data and intermediates in registers, not the whole state
→ lower per-thread register pressure
- Intermediates are exchanged directly between registers via `__shfl_sync()` instead of shared memory
→ less store/reload and synchronization cost

The first optimization is not the computation itself, but distributing one thread's work across the entire warp.

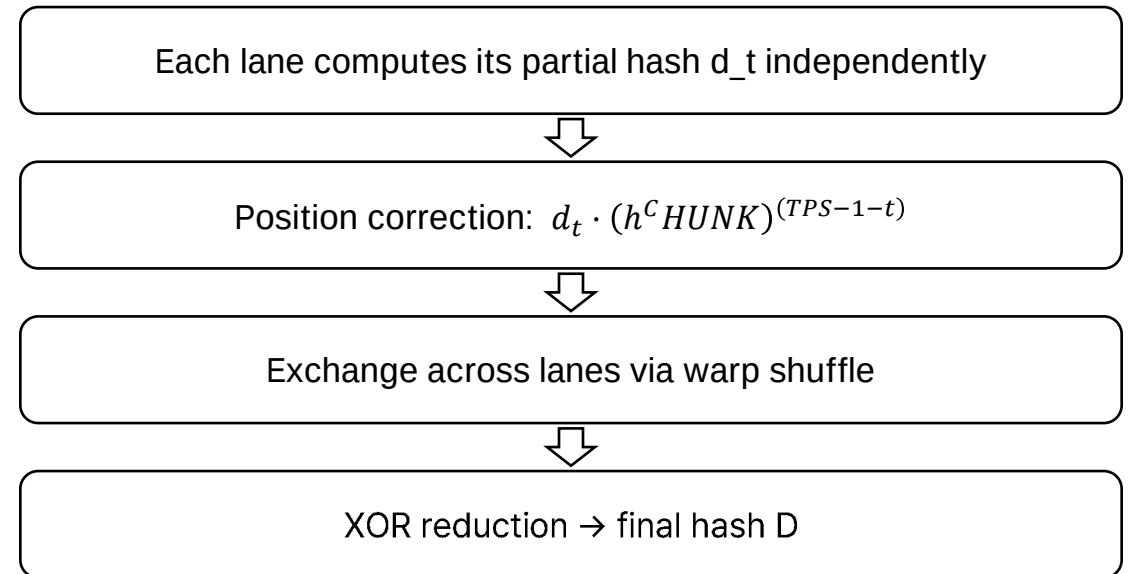
3. Our Implementation Method

❖ Parallelizing the Sequential POLYVAL Hash Chain



$$D = \bigoplus_{t=0}^{TPS-1} d_t \cdot (h^{CHUNK})^{TPS-1-t}$$

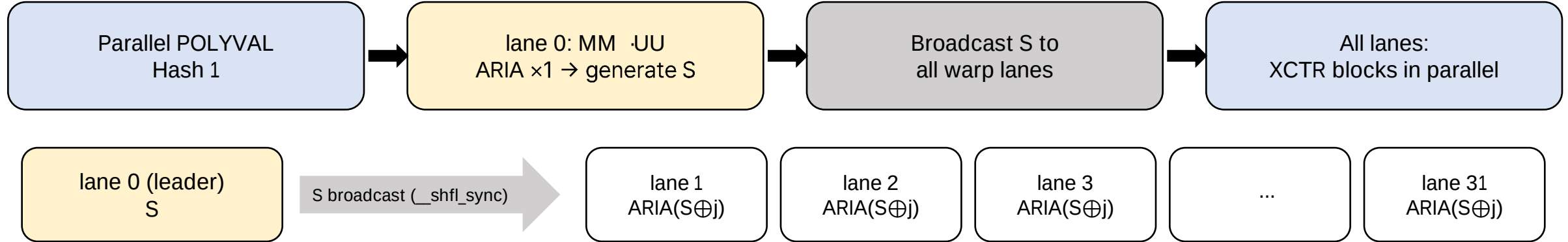
- POLYVAL is a hash chain — each block's result feeds the next — so the basic implementation processes all 1,024 blocks in order
- But POLYVAL is linear over $GF(2^{128})$: split the message into TPS chunks and let each lane compute a partial hash d_t independently
- Each d_t belongs to a different position, so multiply by h raised to the number of remaining blocks to correct it before combining
- Corrected values are exchanged by warp shuffle and merged with XOR → identical to the sequential reference for $TPS = 2 \cdot 4 \cdot 8 \cdot 16 \cdot 32$



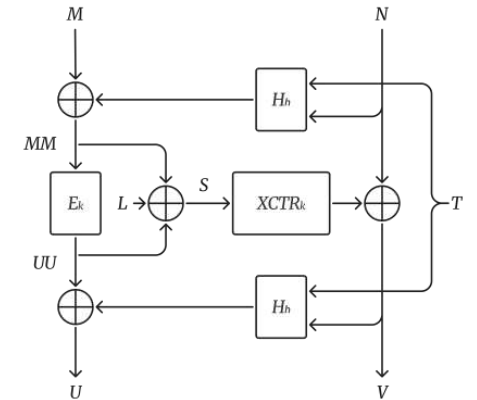
Exploiting POLYVAL's linearity parallelizes the sequential hash chain while preserving correctness.

3. Our Implementation Method

❖ Central ARIA: The Leader Lane Computes Once and Broadcasts to the Warp



- After Hash 1, the lanes' partial hashes are combined into the final hash, yielding $MM \cdot UU$ and one central ARIA that generates S
- The central ARIA runs once per message and is hard to split, so lane 0 (the leader) performs it
- Lane 0's S is broadcast to the whole warp via `__shfl_sync()` — passing register values directly instead of round-tripping shared memory, cutting memory traffic
- Each lane then computes $ARIA(S \oplus j)$ for its counter to produce distinct XCTR blocks
→ every lane rejoins the heavy XCTR work



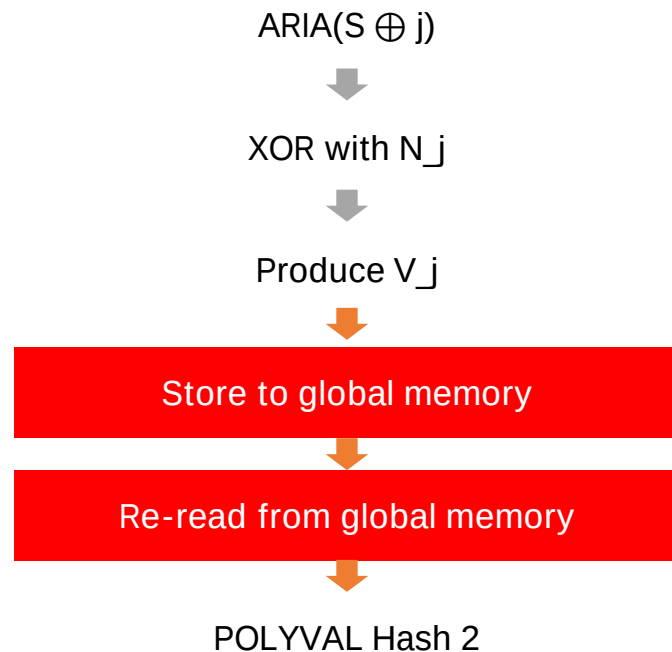
$$XCTR_k(S) = E_k(S \oplus \text{bin}(1)) \parallel E_k(S \oplus \text{bin}(2)) \parallel E_k(S \oplus \text{bin}(3)) \parallel \dots$$

The hard-to-parallelize central ARIA is done once by the leader, and sharing its result lets every lane take part in XCTR.

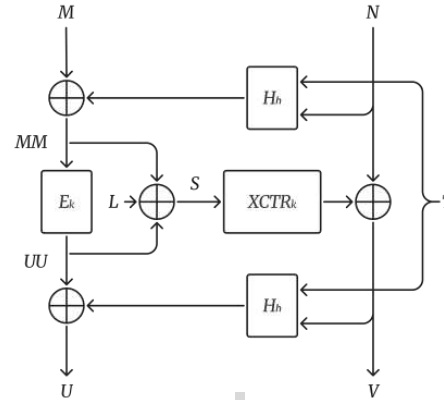
3. Our Implementation Method

- ❖ Use the XCTR result directly in Hash 2 without re-reading it

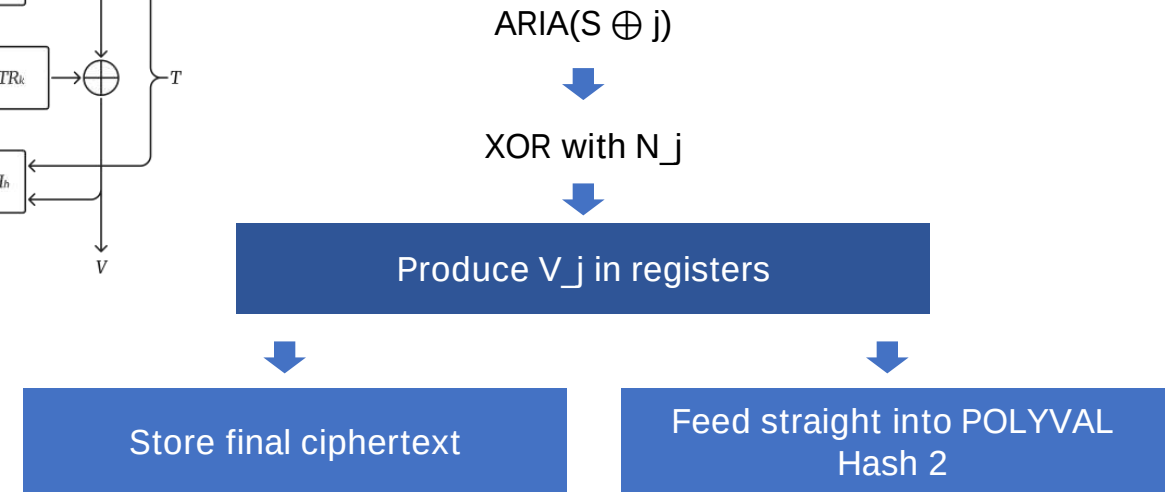
Non-fused approach



Unnecessary
intermediate data
movement



Fused kernel



- Process XCTR and Hash 2 in a single kernel loop
- The intermediate ciphertext is never re-read from global memory
- V_j is fed into POLYVAL while still in registers
- Cuts data-movement cost, not compute

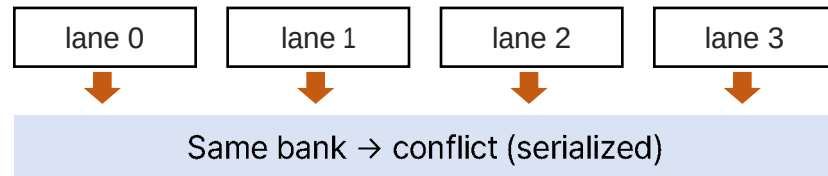
The key to the second optimization is not to compute less, but to avoid re-fetching already-computed values from memory.

3. Our Implementation Method

❖ Per-lane replication + 4-bit split: no conflicts, fits in 48 KiB

- Problem / data-dependent table lookup

Each lane uses a different index into one shared table → different addresses in the same bank → bank conflict (serialized access)



RF=32 per-lane column replication

- ARIA: `tS[256][32]`
- POLYVAL: `table[nibble][word][32]`

lane 0 → column 0 → bank 0

lane 1 → column 1 → bank 1

⋮

lane 31 → column 31 → bank 31

bank = $(32 \times \text{index} + \text{lane}) \bmod 32 = \text{lane}$

- Cost of replication / shared-memory capacity

Replicate the 8-bit table 32× : $256 \times 16 \text{ B} \times 32 = 128 \text{ KiB}$

→ hard to place in shared memory

4-bit table split

$v = (\text{hi} \ll 4) \text{ XOR } \text{lo}$, $\text{M8}[v] = \text{M4a}[\text{hi}] \text{ XOR } \text{M4b}[\text{lo}]$

one 256-entry 8-bit table → two 16-entry 4-bit tables

$2 \times 16 \times 4 \times 32 \times 4 \text{ B} = 16 \text{ KiB}$

128 KiB → 16 KiB (8× smaller)

ARIA S-box 32 KiB

POLYVAL 16 KiB

Total shared memory = 48 KiB / block

- Shared-load bank conflicts in the repeated hot loop (Nsight Compute)

RF = 1

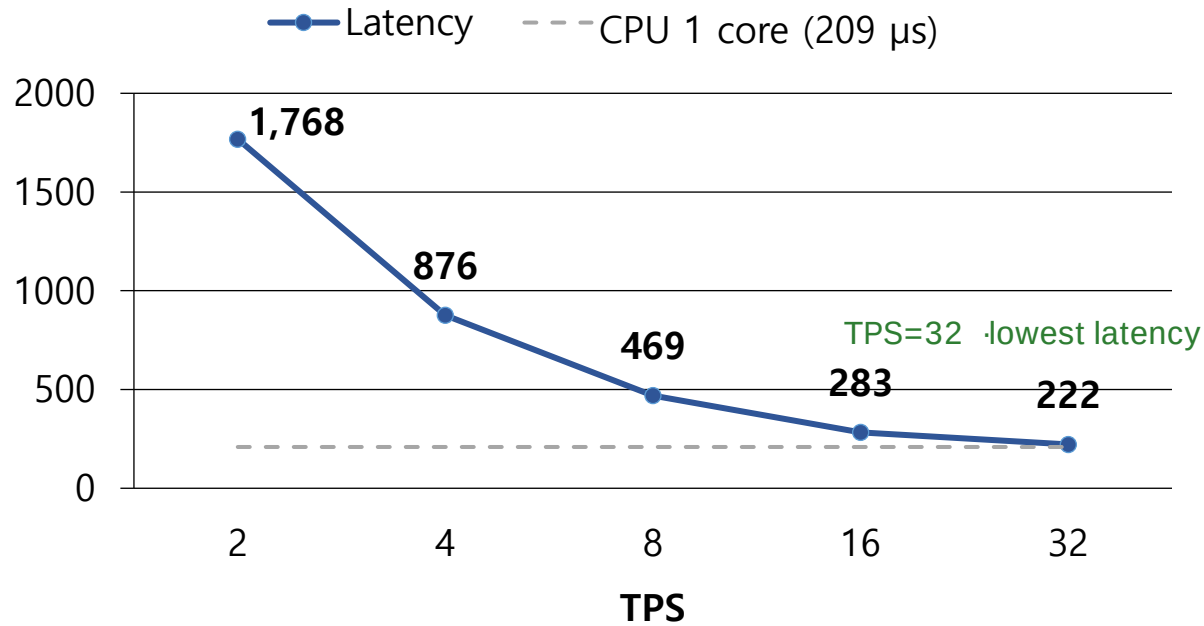
RF = 32

0

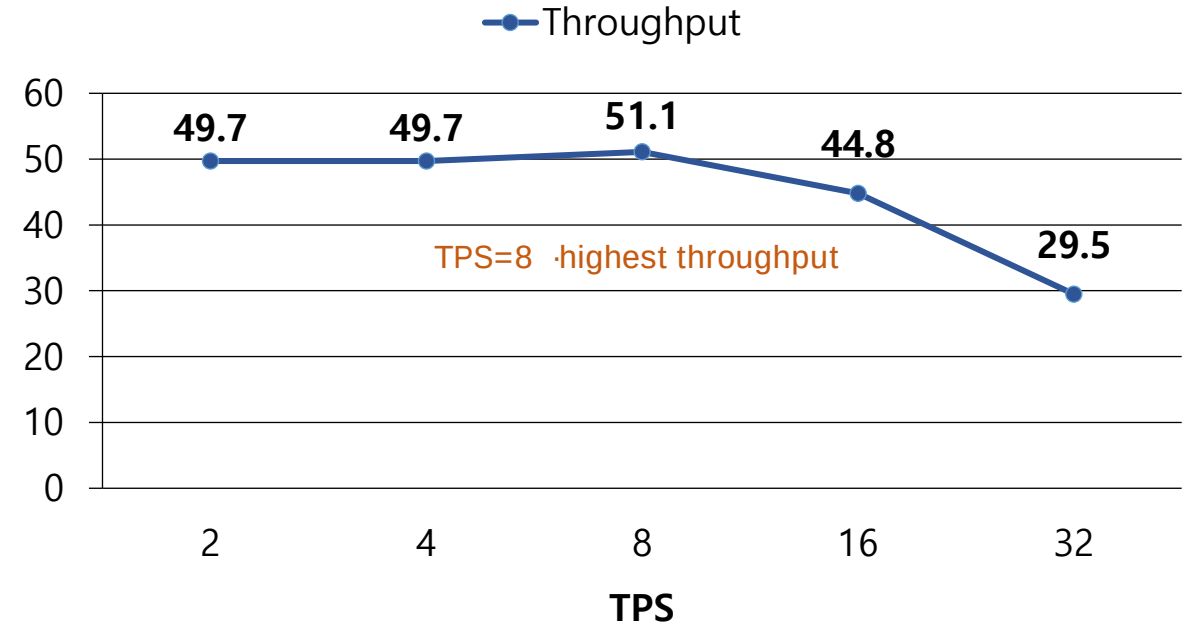
199,409,216

4. Performance

Single-message latency ($\mu\text{s}/\text{message}$)



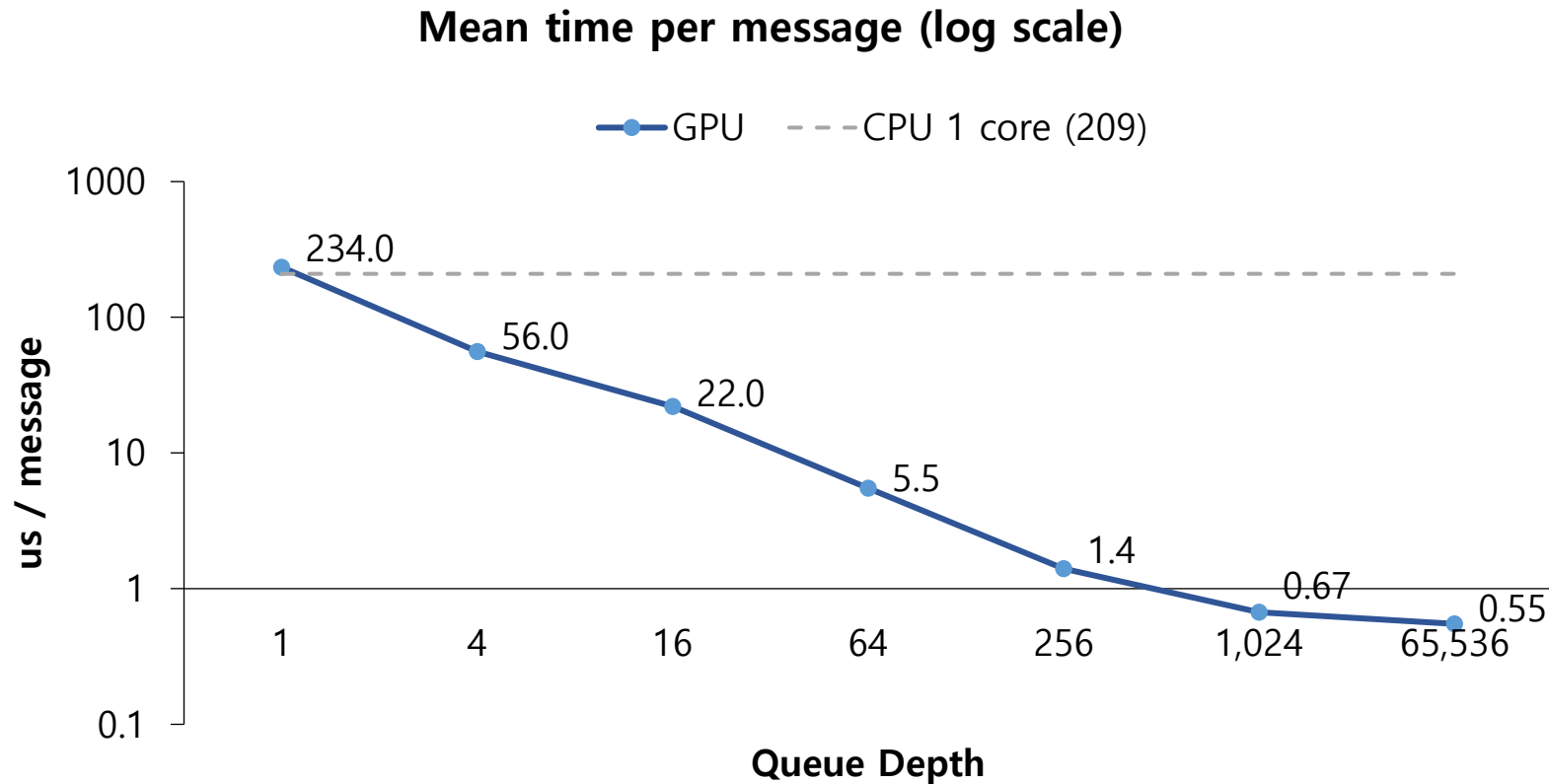
Saturated throughput (GB/s)



warp configuration
TPS=32 $\rightarrow$ 1 msg/warp / TPS=16 $\rightarrow$ 2 msg/warp / TPS=8 $\rightarrow$ 4 msg/warp

Higher TPS speeds up a single message, lower TPS increases total throughput

4. Performance



- **QD1**
GPU 234 μ s vs CPU 209 μ s
→ Comparable for a single message

- **QD4**
GPU avg 56 μ s/message
→ ~3.7 $\times$ throughput vs CPU sequential

- **Saturated**
~0.55 μ s/message
~ 30 GB/s

At QD1 the GPU matches the CPU, from QD4 it leads on throughput

* QD4's 56 μ s/message is not the latency of a single message, but the average processing cost — the total time for 4 messages divided by 4.
ARIA-256, TPS=32, SPB=8, 16 KiB/message. CPU baseline dedicated latency $\approx$ 209 μ s ($\approx$ 213 μ s in the QD sweep raw CSV).

결론 및 향후 연구

- Optimizations Applied
 - We applied warp-cooperative parallelization exploiting the linearity of POLYVAL, XCTR-Hash 2 fusion, a bank-conflict-free shared table, and TPS SPB tuning.
- Performance Results
 - GPU processing time for a single 16 KiB message was reduced from about 2,310 μ s to 222 μ s, reaching parity with the 209 μ s of one CPU core. When enough messages are available, TPS=8 achieved a throughput of about 51 GB/s.
- Key Findings
 - In GPU cipher implementations, performance is determined not only by the amount of computation but also by how work is mapped to threads, unnecessary data movement, shared-memory bank conflicts, and TPS SPB selection matched to the workload.

Thank You